Inżynieria wymagań

Członkowie zespołu:

1. Macierz kompetencji zespołu

Kompetencje	Mateusz Aleksander	Mateusz Basta	Mikołaj Zajdek
Programowanie Python	Posiada	Posiada	Posiada
Technologie Webowe	Posiada	Posiada (podstawy)	Posiada
Bazy danych	Posiada	Posiada (podstawy)	Nie posiada
Znajomość UML	Nie posiada	Posiada	Posiada (podstawy)
Testowanie oprogramowania	Posiada (podstawy)	Posiada (podstawy)	Posiada
Obsługa Gita	Posiada	Posiada (podstawy)	Posiada

2. Pytania oraz odpowiedzi do zmodelowania projektu:

Pytanie	Odpowiedź	Uwagi
Jak przebiega proces autoryzacji i identyfikacji pracowników w systemie?	Pracownik skanuje kod QR w przeglądarce, wykonywana jest krótka sekwencja klatek z kamery. Następnie system sprawdza liveness (mrugnięcie), ważność QR (mvp) i porównuje twarz z wektorem twarzy w bazie danych (mvp). Każda próba kończy się zapisem zdarzenia (mvp).	Weryfikacja działa wyłącznie dla użytkownika powiązanego z danym QR. Przy nieudanej próbie weryfikacji zostaje wykonane oraz zapisane zdjęcie ze zdarzenia.
Co ma zapewnia system?	Automatyzację rejestracji wejść i wyjść z weryfikacją tożsamości (QR + twarz + liveness) (mvp), panel administracyjny do zarządzania pracownikami i raportami (mvp).	
Jaką mamy zapewnić skuteczność wykrywania twarzy?	Skuteczność biblioteki wynosi 99% według dokumentacji biblioteki > 90% (mvp). W projekcie użyto face_recognition z progiem podobieństwa 0.6. Skuteczność	

	zależy od jakości obrazu, oświetlenia i ustawienia twarzy.	
Jak ma wyglądać frontend aplikacji?	Strona główna: podgląd kamery, automatyczne skanowanie QR (mvp) , wybór kierunku (wejście/wyjście), komunikaty statusu. Panel admina: logowanie, zarządzanie pracownikami, raporty, eksport PDF (mvp).	
Rodzaj kamery?	Dowolna kamera dostępna w przeglądarce (mvp) (USB/wbudowana).	
Jakie mają być funkcjonalności panelu admina?	Dodawanie pracowników z obrazem twarzy (plik lub zdjęcie z kamery) (mvp), generowanie i pobieranie QR (mvp), edycja daty ważności QR (mvp), usuwanie pracowników, podgląd zdarzeń i raporty PDF (mvp).	Hasło admina jest haszowane i przechowywane w bazie.
Co mają zawierać raporty?	Zdarzenia wejść/wyjść z datą, statusem, kierunkiem, pracownikiem oraz kodem błędu (mvp), możliwość podglądu zdjęcia próby dla błędów (mvp). PDF zawiera listę oraz obrazy (mvp).	Raporty są generowane dynamicznie z bazy danych.
Jaka będzie liczba użytkowników?	System dla dużej fabryki (około 50 os.) (mvp)	Ograniczeniem praktycznym jest SQLite i zasoby serwera.

Czy wszystkie wymagania klienta są możliwe do spełnienia? TAK, system realizuje weryfikację QR + twarz + liveness oraz panel admina i raporty.

3. Ustalony format danych wejściowych:

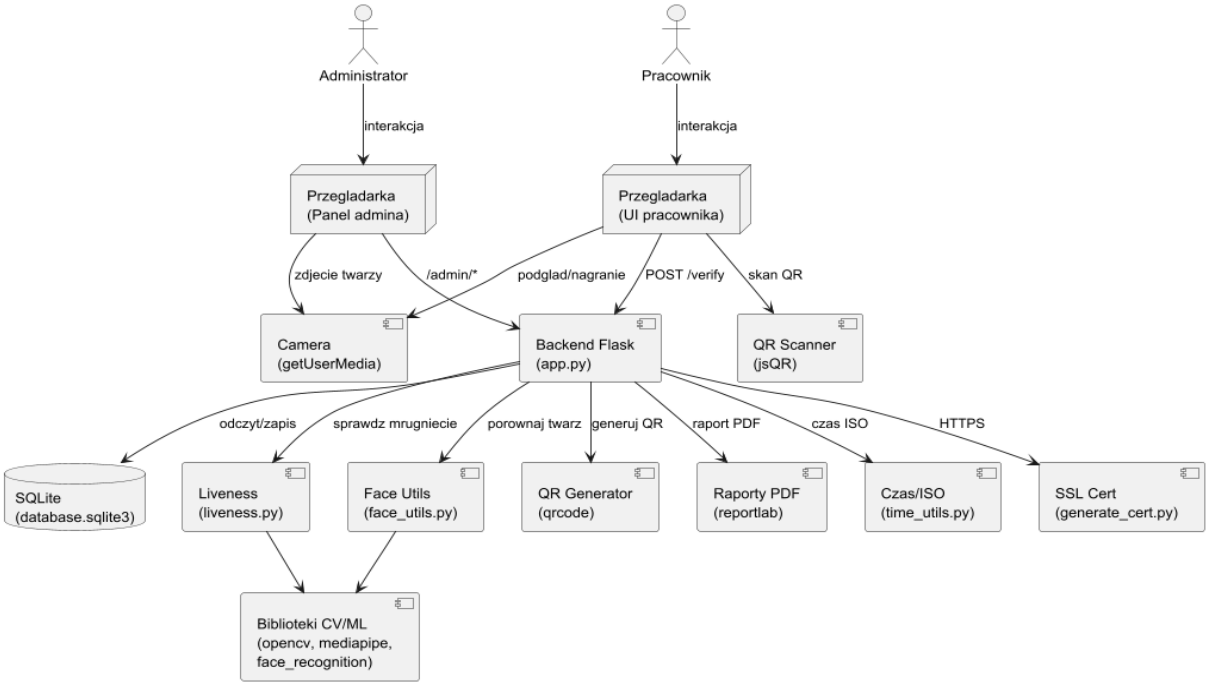
Autoryzacja	Użytkownicy	Wejścia/Wyjścia	Raporty
qr_code (string)	id (int)	id (int)	zakres dat (start/end)
frames (lista base64)	first_name, last_name, name (string)	user_id (int/NULL)	lista zdarzeń
direction (IN/OUT)	face_encoding (JSON)	timestamp (ISO)	status, kierunek, pracownik
	qr_code (EMP:{id})	status (OK/FAIL)	kod błędu, obraz
	qr_expires_at (ISO)	error_code, qr_code (int), (EMP:{id})	PDF jako plik wynikowy
	created_at, updated_at (ISO)	attempt_image_b64	

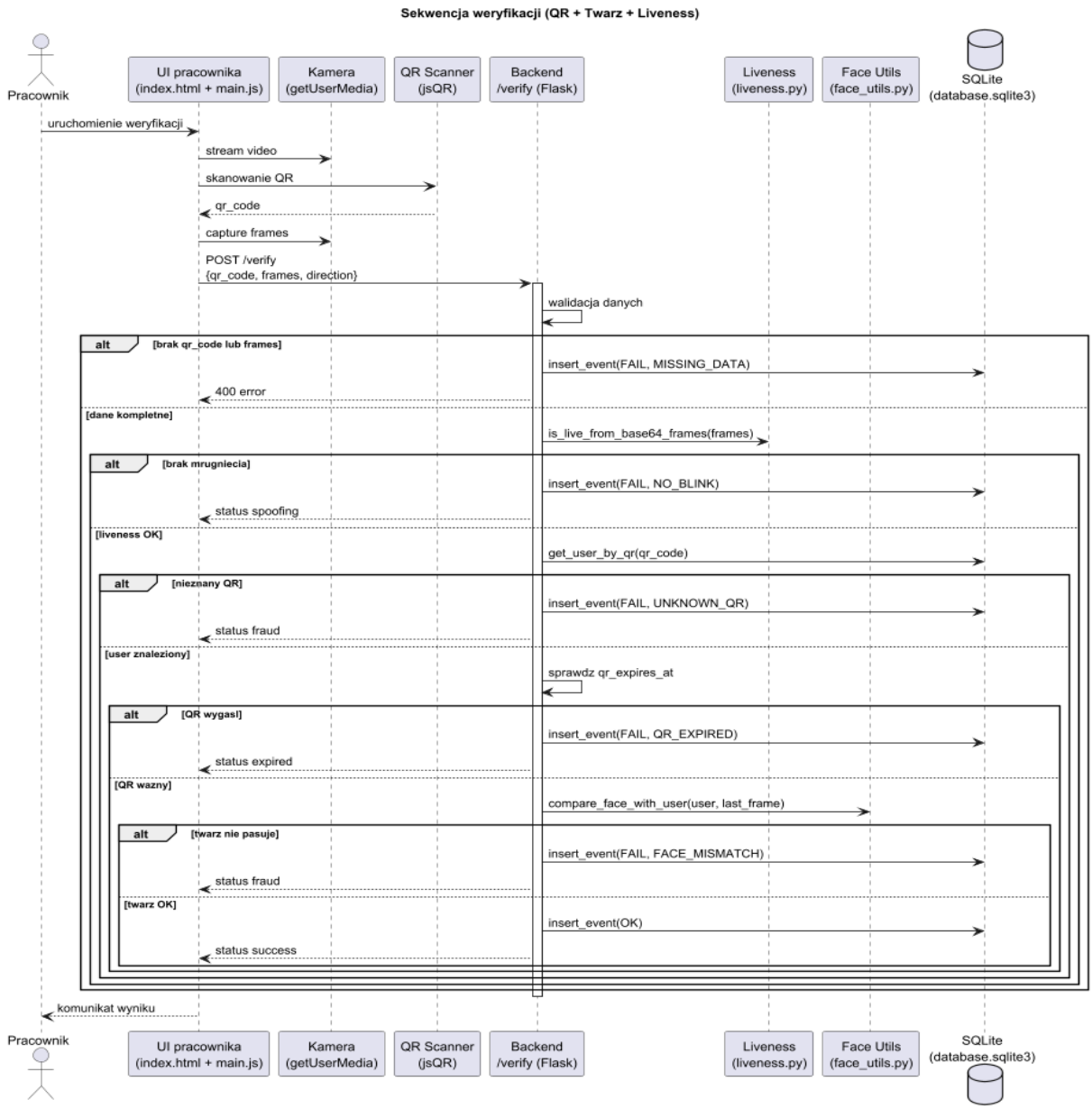
4.1. Modelowanego systemu za pomocą tabeli:

Aktorzy	Pracownicy, Administrator.
Opis	System weryfikuje wejścia/wyjścia na podstawie kodu QR oraz rozpoznawania twarzy z testem żywotności (mrugnięcie). Administrator zarządza pracownikami i raportami.
Dane	Kod QR, klatki wideo, wektor twarzy, czas zdarzenia, status i kod błędu, hash hasła.
Wyzwalacz	Zeskanowanie kodu QR i uruchomienie weryfikacji w UI albo działania admina dodanie/edycja/usunięcie pracownika, generowanie raportu.
Odpowiedź	Wynik weryfikacji z komunikatem i zapisem zdarzenia. Dla błędów: kod przyczyny (np. brak danych, brak mrugnięcia, nieznany QR, niezgodność twarzy, wygasły QR).
Uwagi	Weryfikacja liveness bazuje na detekcji mrugnięcia z wielu klatek.

4.2. Przedstawienie modelowanego systemu za pomocą diagramów UML:

Architektura systemu Face-QR Auth





5. Język implementacji: Python, JS

Uzasadnienie: Projekt jest zrealizowany w Pythonie Flask z użyciem bibliotek do rozpoznawania twarzy face-recognition, przetwarzania obrazu opencv-python, liveness mediapipe, generowania QR qrcode i raportów PDF reportlab. Pythona umożliwia szybkie prototypowanie i łatwą integrację z bazą SQLite.